

Google Developer Group
Universitas Sriwijaya

Learning 11

Global State Management

Sharing data across components



```
const org = filterByOrg ? study.lead_organization === filterByOrg : true  
const status = filterByStatus ? study.status === filterByStatus : true  
const matchStatus = filterByStatus ? study.status === filterByStatus : true  
const matchStatus = filterByStatus ? study.status === filterByStatus : true
```

```
function filterStudies({ studies, filterByOrg, filterByStatus }) {  
  return studies.filter(study => {  
    const org = filterByOrg ? study.lead_organization === filterByOrg : true  
    const status = filterByStatus ? study.status === filterByStatus : true  
    return org & status  
  })  
}
```

Local State vs Shared State

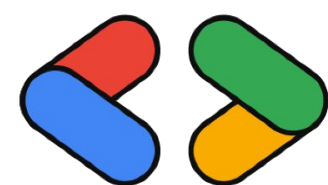
In React, most state lives inside components.

```
function StudentsPage() {  
  const [students, setStudents] = useState([])  
}
```

This works well for local UI state.

However, some data must be used by many components.

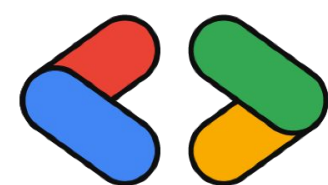
Examples: authentication user, theme (dark / light)



The Props Drilling Problem

When multiple components need the same data, we often pass it via props.

```
<App students={students}>  
  <Layout students={students}>  
    <Dashboard students={students}>  
      <StudentList students={students}/>  
    </Dashboard>  
  </Layout>  
</App>
```

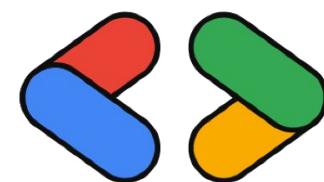


Solution: Context API

React provides Context API to share data globally.

Context allows components to:

- access shared state
- without passing props manually



Core Concepts of Context API

1. Context

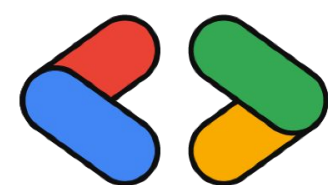
Creates the shared container

```
const ThemeContext = createContext( )
```

2. Provider

Supplies data to the component tree

3. Consumer/useContext



Creating a Context

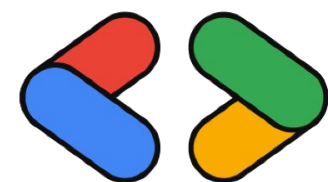
```
import { createContext } from "react"

export const ThemeContext = createContext()
```

The context acts as a shared data channel.

But by itself, it does not provide data yet.

We need a Provider

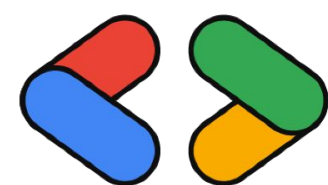


Using a Provider

Provider wraps part of the application

```
<ThemeContext.Provider value={theme}>  
  <App />  
</ThemeContext.Provider>
```

Now every component inside <App /> can access theme.



Accessing Context with useContext

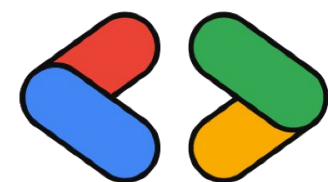
Components can access shared data using useContext.

```
import { useContext } from "react"

const theme = useContext(ThemeContext)
```

This allows the component to read the global state.

No props required.

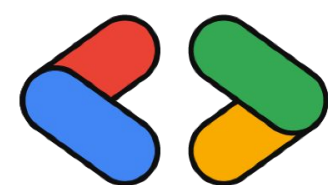


Example Use Cases

Common scenarios for Context API:

- theme management
- authentication user
- language settings
- shared application data

Important note: Context API is useful for global or widely shared state.

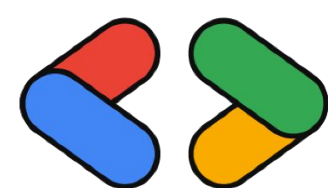


Google Developer Group
Universitas Sriwijaya

Challenge! (FE Member Only*)

1. Buat sebuah web (Tema bebas, disarankan melanjutkan project UTS)
2. Implementasikan minimal dua buah Context — satu untuk tema (dark/light mode) dan satu untuk data yang diakses antar komponen
3. Implementasikan apa yang telah dipelajari
4. Lakukan pull request dan masukkan link repo github kamu di folder /11-global-state-management/task.txt
5. Deadline 19 Juni 2026

Selamat Mencoba ~



Google Developer Group
Universitas Sriwijaya



Thank You !